Hacettepe University

Department of Electrical&Electronics Engineering

ELE432 – Advanced Digital Design

Preliminary Work – Advanture Game

2210357093

Haydar Deniz Korkmaz

1. Time Spent

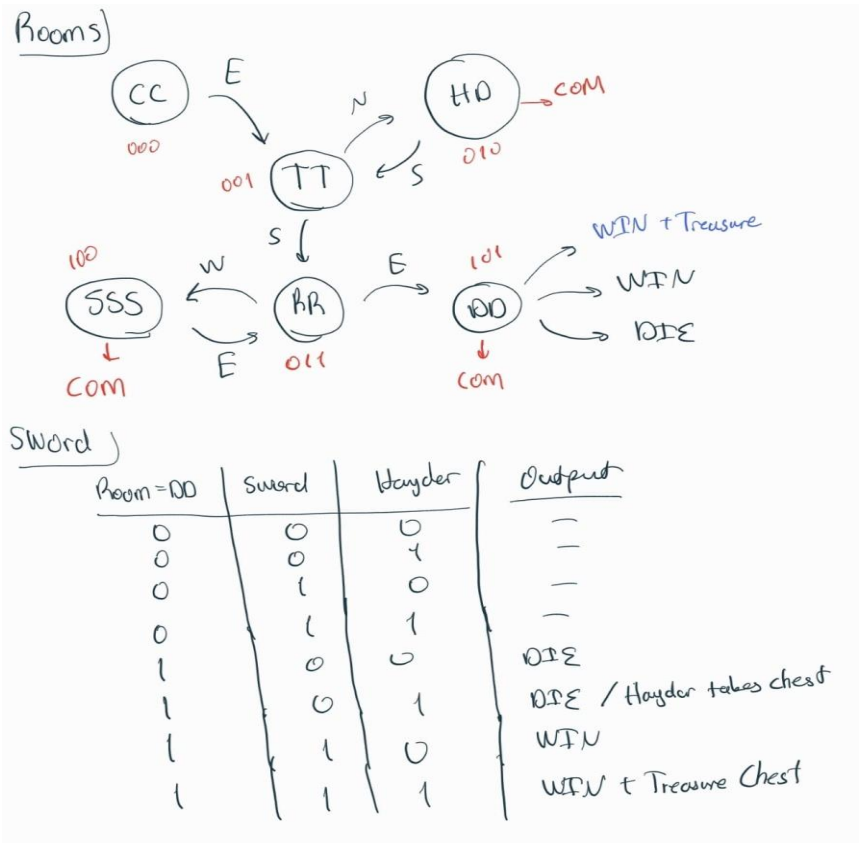
Approximately 4 hours were spent on this lab. Most of the time was spent designing the FSMs, debugging the SystemVerilog code, and running simulations in Questa but also preparing report.

2. State Transition Diagrams

As an extension to the original adventure game, an additional room called Haydar's Dorm was added to the map. This room is located to the north of the Twisty Tunnel. When the player enters Haydar's Dorm, a new companion character named Haydar joins the player. This required adding a new state (HD) to the Room FSM and a new register signal (Haydar_with_us) to keep track of whether Haydar has joined the player.

The game logic was also extended in the Inventory FSM to include new outcomes when entering the Dragon's Den. In the original game, the outcome depended only on whether the player had the sword. In the extended version, the outcome depends on both the sword and Haydar. If the player enters the Dragon's Den without the sword and without Haydar, the player dies. If the player has Haydar but no sword, Haydar defeats the dragon but keeps the treasure, so the player loses. If the player has the sword, the player wins. If the player has both the sword and Haydar, the player wins and also obtains the treasure. This extension added additional game logic and made the FSM design more complex and interesting.

Here is the handwriting FSM diagrams.



The Room FSM controls the player's location in the game. The system starts in the Cave of Cacophany. Depending on the direction inputs (north, south, east, west), the FSM transitions between rooms such as Twisty Tunnel, Rapid River, Secret Sword Stash, Haydar's Dorm, and Dragon's Den. From Dragon's Den, the system transitions to either Victory Vault or Grevious Graveyard depending on the inventory FSM outputs.

The Inventory FSM keeps track of whether the player has obtained the sword and whether Haydar is accompanying the player. When the player enters the Dragon's Den, the FSM determines the outcome of the game based on these conditions.

3. Behavioral SystemVerilog Code

Room_fsm.sv

```
module room_fsm(
        input logic clk,
        input logic rst,
        input logic north,
        input logic south,
        input logic east,
        input logic west,

        output logic has_sword,
        output logic Haydar_with_us,
        output logic in_DD

);

        typedef enum logic[2:0]{
                CC = 3'b000 , // Cave of Cacophany
                TT = 3'b001 , // Twisty Tunnel
                HD = 3'b010 , // Haydar's Dorm
                RR = 3'b011  ,// Rapid River
                SSS = 3'b100 ,// Secret Sword Stash
                DD = 3'b101  , // Dragon's Den
                VV = 3'b110 , // Victory Vault
                GG = 3'b111  // Grevious Graveyard
        } room_state;

        room_state state, next_state ;

        always_ff @(posedge clk or posedge rst) begin
                if (rst)
                        state <= CC;
                else
                        state <= next_state;
        end
        // Sword register
        always_ff @(posedge clk or posedge rst) begin
                 if (rst)
                         has_sword <= 0;
                 else if (state == SSS)
                         has_sword <= 1;
        end
```

```

        // Haydar register
        always_ff @(posedge clk or posedge rst) begin
                if (rst)
                        Haydar_with_us <= 0;
                else if (state == HD)
                        Haydar_with_us <= 1;
        end

        always_comb begin
                next_state = state;

                case (state)
    CC: begin
      if (east)
        next_state = TT;
                        end

                 TT: begin
                        if (north)
                                next_state = HD;
                        else if (south)
                                next_state = RR;
                        end

                        HD: begin
                                if (south)
                                        next_state = TT;
                        end

                        RR: begin
                                if (west)
                                        next_state = SSS;
                                else if(east)
                                        next_state = DD;
                        end

                        SSS: begin
                                if (east)
                                        next_state = RR;
                        end
                        endcase
        end

        assign in_DD = (state == DD);

        endmodule

```

İnventory_fsm.sv

```
module inventory_fsm(
	input logic clk,
	input logic rst,
	input logic has_sword,
	input logic Haydar_with_us,
	input logic in_DD,

	output logic win_goto_VV,
	output logic die_goto_GG,
	output logic win_with_treasure,
	output logic Haydar_win_you_lose
);

	typedef enum logic{
		wait_t = 1'b0 ,
		in_Dragon = 1'b1
	} inventory_state_t;

	inventory_state_t state , next_state ;

	always_ff @(posedge clk or posedge rst) begin
			if (rst)
				state <= wait_t;
			else
				state <= next_state;
		end

	always_comb begin
			next_state = state;
			win_goto_VV = 0;
			die_goto_GG = 0;
			win_with_treasure = 0;
			Haydar_win_you_lose = 0;
			case (state)
				wait_t:  begin
					if (in_DD == 1)
						next_state = in_Dragon ;
				end
				in_Dragon: begin
					if ((has_sword == 0)&&(Haydar_with_us == 0))
						die_goto_GG  = 1 ;
					else if ((has_sword == 0)&&(Haydar_with_us == 1))
						Haydar_win_you_lose = 1;
					else if ((has_sword == 1)&&(Haydar_with_us == 0))
						win_goto_VV = 1 ;
```

```
                                        else if ((has_sword == 1)&&(Haydar_with_us == 1))
                                                win_with_treasure = 1 ;
                                end
                        endcase
                        end
        endmodule
```

**********This codes are also available in Github repo in this link.**********

4. Testbench

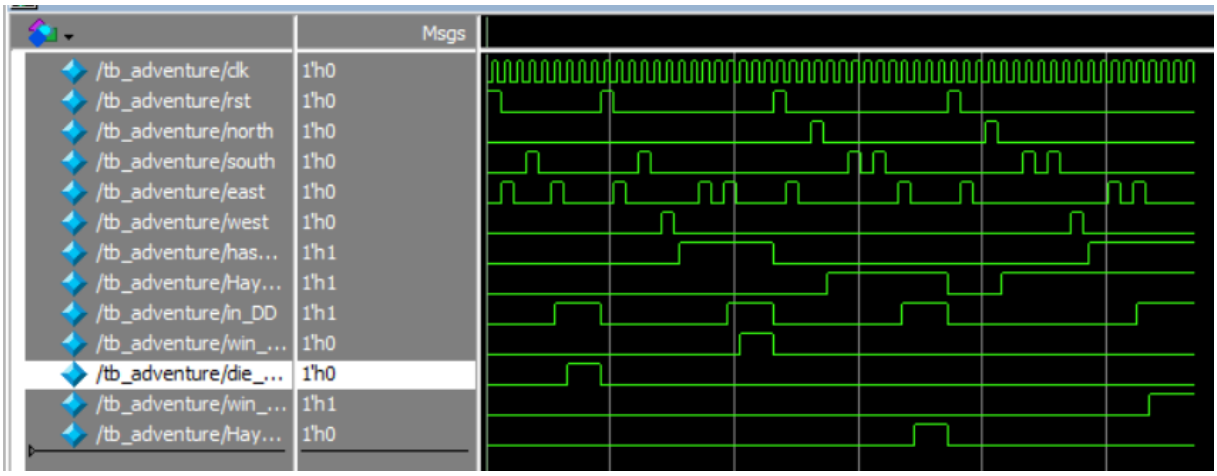
Testbench Code:

```
module tb_adventure;

  logic clk;
  logic rst;
  logic north;
  logic south;
  logic east;
  logic west;

  logic has_sword;
  logic Haydar_with_us;
  logic in_DD;

  logic win_goto_VV;
  logic die_goto_GG;
  logic win_with_treasure;
  logic Haydar_win_you_lose;

  // room_fsm instance
  room_fsm u_room (
    .clk(clk),
    .rst(rst),
    .north(north),
    .south(south),
    .east(east),
    .west(west),
    .has_sword(has_sword),
    .Haydar_with_us(Haydar_with_us),
    .in_DD(in_DD)
  );

  // inventory_fsm instance
  inventory_fsm u_inventory (
    .clk(clk),
    .rst(rst),
    .has_sword(has_sword),
    .Haydar_with_us(Haydar_with_us),
    .in_DD(in_DD),
    .win_goto_VV(win_goto_VV),
    .die_goto_GG(die_goto_GG),
    .win_with_treasure(win_with_treasure),
```

```
    .Haydar_win_you_lose(Haydar_win_you_lose)
  );

  // clock generation
  initial begin
    clk = 0;
    forever #5 clk = ~clk;
  end

  // test sequence
  initial begin
    // başlangıç değerleri
    rst = 1;
    north = 0;
    south = 0;
    east = 0;
    west = 0;

    #12;
    rst = 0;

    // SENARYO 1: sword almadan dragon'a git -> die_goto_GG
    pulse_east();   // CC -> TT
    pulse_south();  // TT -> RR
    pulse_east();   // RR -> DD

    #20;

    // yeniden reset
    rst = 1;
    #10;
    rst = 0;

    // SENARYO 2: sword al, sonra dragon -> win_goto_VV
    pulse_east();   // CC -> TT
    pulse_south();  // TT -> RR
    pulse_west();   // RR -> SSS
    #10;          // sword register için 1 clock bekle
    pulse_east();   // SSS -> RR
    pulse_east();   // RR -> DD

    #20;

    // yeniden reset
    rst = 1;
    #10;
    rst = 0;

    // SENARYO 3: Haydar al, sword alma -> Haydar wins, you lose
    pulse_east();   // CC -> TT
    pulse_north();  // TT -> HD
    #10;          // Haydar register için 1 clock bekle
    pulse_south();  // HD -> TT
    pulse_south();  // TT -> RR
    pulse_east();   // RR -> DD

    #20;
```

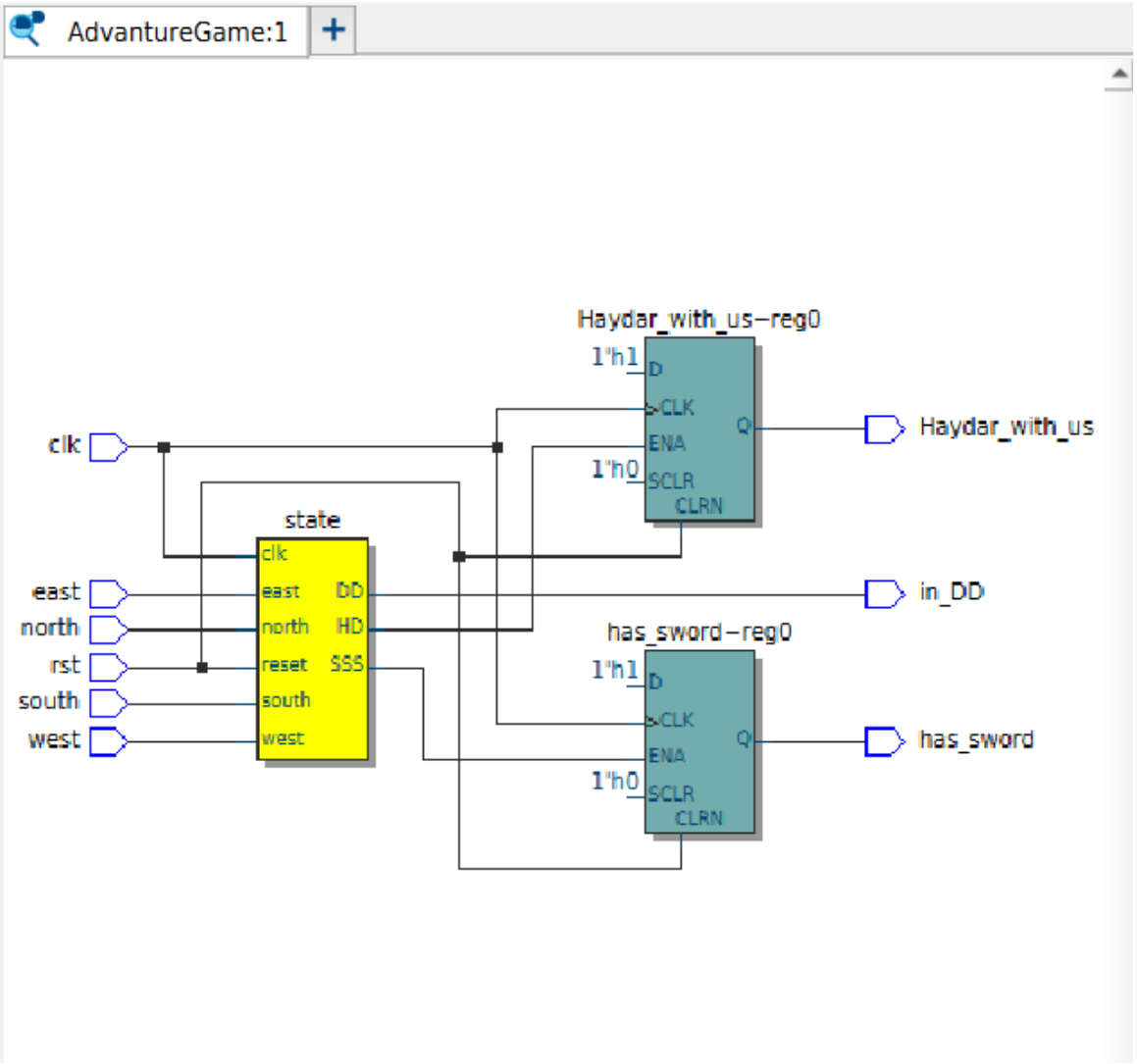
```
    // yeniden reset
    rst = 1;
    #10;
    rst = 0;

    // SENARYO 4: hem Haydar hem sword -> treasure win
    pulse_east();   // CC -> TT
    pulse_north();  // TT -> HD
    #10;
    pulse_south();  // HD -> TT
    pulse_south();  // TT -> RR
    pulse_west();   // RR -> SSS
    #10;
    pulse_east();   // SSS -> RR
    pulse_east();   // RR -> DD

    #30;
    $stop;
  end

  // yön pulse taskları
  task pulse_north;
    begin
      north = 1; #10; north = 0; #10;
    end
  endtask

  task pulse_south;
    begin
      south = 1; #10; south = 0; #10;
    end
  endtask

  task pulse_east;
    begin
      east = 1; #10; east = 0; #10;
    end
  endtask

  task pulse_west;
    begin
      west = 1; #10; west = 0; #10;
    end
  endtask

endmodule
```

5. Waveforms

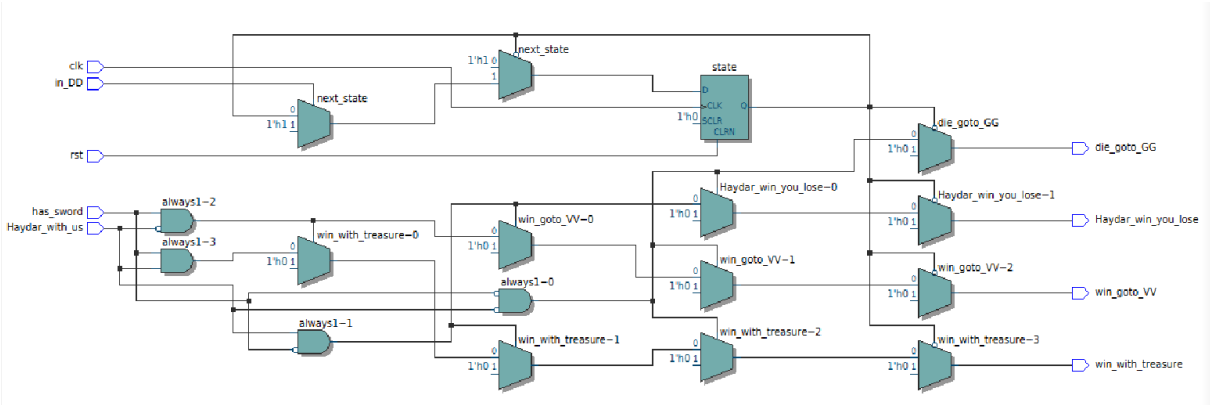


6. RTL Schematics

Room:



Inventory:



7. Extra Credits

For the extra credit part of the assignment, the adventure game was extended by adding a new room called Haydar's Dorm, which is located to the north of the Twisty Tunnel. When the player enters this room, a companion character named Haydar joins the player. This required adding a new state to the Room FSM and a new register signal to keep track of whether Haydar is accompanying the player.

The game logic was also extended in the Inventory FSM. In the original game, the outcome in the Dragon's Den depended only on whether the player had the sword. In the extended version, the outcome depends on both the sword and Haydar. If the player enters the Dragon's Den without the sword and without Haydar, the player dies and goes to the Graveyard. If the player has Haydar but no sword, Haydar defeats the dragon but keeps the treasure, so the player loses only Haydar wins. If the player has the sword, the player wins and goes to the Victory Vault. If the player has both the sword and Haydar, the player wins and also obtains the treasure. This extension increased the complexity of the FSM design and added more interesting game behavior.